



IILM UNIVERSITY, GREATER NOIDA
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

INTERNSHIP REPORT

SUBMITTED BY

Name: RUDRA PRATAP
Roll No: 25SCS1003002676
Class: 2CSE1

PROJECT TITLE

FORECASTING OF SMART CITY TRAFFIC PATTERNS

DOMIAN

DATA SCIENCE AND MACHINE LEARNING

DURATION

FROM 29 JULY 2026 TO 24 AUGUST 2026



Industrial Internship Report on "Forecasting of Smart City Traffic Patterns"

Prepared by: [Rudra Pratap]

Host Organization: UniConverge Technologies Pvt. Ltd. (UCT) in collaboration with upSkill Campus & The IoT Academy

Executive Summary

This report provides details of the Industrial Internship provided by upSkill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was **Forecasting of Smart City Traffic Patterns**. The core objective was to develop an intelligent machine learning pipeline capable of predicting hourly traffic volumes across four key city junctions. By incorporating temporal extraction, calendar event analysis (holidays and weekends), cyclical encoding, and autoregressive lag/rolling features into an XGBoost Regressor model, the system accurately forecasts traffic spikes to assist city authorities in proactive congestion management and urban infrastructure planning.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solutions for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

- 1. Preface**
- 2. Introduction**
 - 2.1 About UniConverge Technologies Pvt Ltd
 - 2.2 About upskill Campus
 - 2.3 Objective
 - 2.4 Reference
 - 2.5 Glossary
- 3. Problem Statement**
- 4. Existing and Proposed Solution**
- 5. Proposed Design/ Model**
 - 5.1 High Level Diagram

- 5.2 Low Level Diagram
- 5.3 Interfaces & Feature Engineering

6. Performance Test

7. My Learnings

8. Future Work Scope

1. Preface

Over the course of 6 weeks, this internship provided an intensive hands-on experience in applying data science and machine learning techniques to real-world smart city infrastructure challenges. In modern urban environments, rapid population growth and increasing vehicle density lead to significant traffic congestion, economic loss, and environmental pollution.

Relevant internships are critical for career development as they bridge the gap between theoretical academic knowledge and production-grade engineering practices. Working under the guidance of industrial mentors from UCT allowed me to understand how time-series data is ingested, engineered, modeled, and evaluated in a real-world setting.

The project focuses on building an automated time-series forecasting engine to predict hourly vehicle counts at four city junctions, ensuring city administrators can anticipate traffic surges on regular workdays, weekends, and national holidays.

The internship program followed a structured 6-step workflow:

1. **Explore Problem Statement & About UCT:** Deep-dive into smart city traffic data and organizational ecosystem.
2. **Follow Project Instructions & Plan a Solution:** Define time-series modeling strategies and feature extraction methods.
3. **Work on Project:** Clean data, construct lag/rolling features, and build XGBoost models.
4. **Continue Work & Check for Improvements:** Tune hyperparameters, test cyclical variables, and reduce cross-junction RMSE.
5. **Validate Implementation & Performance:** Validate using out-of-time test sets and compute evaluation metrics (RMSE, MAE, R^2).
6. **Submit Project & Report, Get Certified:** Document deliverables and present outcomes.

I extend my sincere gratitude to the technical mentors at UniConverge Technologies Pvt Ltd and upSkill Campus for their continuous feedback and guidance throughout this project.

Message to Juniors & Peers: Focus heavily on understanding the underlying domain logic and data preprocessing pipelines. In machine learning engineering, feature engineering and proper evaluation setup often yield significantly higher performance gains than simply tuning hyperparameter grids.

2. Introduction

2.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) was established in 2013, operating in the Digital Transformation domain and providing Industrial solutions with a prime focus on sustainability and Return on Investment (RoI).

To build cutting-edge solutions, UCT leverages technologies such as the Internet of Things (IoT), Cyber Security, Cloud Computing (AWS, Azure), Machine Learning, Communication Protocols (4G/5G/LoRaWAN), and Full-Stack Web Development.

Key platform offerings include:

- **UCT Insight:** An Industrial IoT platform built with Java backend and ReactJS frontend, supporting MySQL and NoSQL databases. It handles device connectivity via MQTT, CoAP, HTTP, Modbus TCP, and OPC UA, featuring dynamic dashboards, analytics, rule engines, and third-party integrations (Power BI, SAP).
- **Factory Watch:** A modular Smart Factory SaaS platform enabling scalable production monitoring, Overall Equipment Effectiveness (OEE) metrics, and asset digital twins.
- **LoRaWAN Solutions:** Applied in Agritech, Smart Cities, Smart Metering (Water/Gas/Electricity), and Intelligent Street Lighting.
- **Predictive Maintenance:** Machine health monitoring utilizing embedded sensors and predictive ML models to estimate Remaining Useful Life (RUL) of industrial machinery.

2.2 About upSkill Campus & The IoT Academy

upSkill Campus (USC) is an executive career development platform delivering self-paced learning, industry mentorship, and practical internship programs. USC aims to upskill 1 million learners over 5 years. The IoT Academy serves as the EdTech division of UCT, running executive certification programs in collaboration with EICT Academies at premier institutions like IIT Kanpur, IIT Roorkee, and IIT Guwahati.

2.3 Objective

The primary objectives of this industrial internship program were:

- Gain practical experience applying data science methodologies to smart-city IoT datasets.
- Address urban mobility and traffic management challenges faced by civic authorities.
- Enhance technical competency in time-series feature engineering, gradient boosting algorithms, and model validation.
- Develop essential professional skills including code optimization, analytical problem-solving, and technical documentation.

2.4 Reference

1. **Repository & Code Base:** Project implementation script using Python, Pandas, XGBoost, and Scikit-Learn.
2. **Dataset Source:** Smart City Traffic Patterns Dataset (train_aWnotuB.csv).
3. **XGBoost Documentation:** Gradient Boosting algorithms for tabular time-series forecasting.

2.5 Glossary

Term	Acronym / Definition
UCT	UniConverge Technologies Pvt Ltd
USC	upSkill Campus
RMSE	Root Mean Squared Error (evaluates large prediction errors penalization)
MAE	Mean Absolute Error (evaluates average magnitude of errors)
R^2	Coefficient of Determination (proportion of variance explained by model)
Lag Feature	Historical values of target variable at fixed previous time intervals ($t - 1, t - 24$, etc.)

Term	Acronym / Definition
Cyclical Encoding	Transforming cyclical temporal features (hours, months) into sine and cosine components

3. Problem Statement

Modern municipal authorities require an intelligent traffic management infrastructure to transition into a Smart City. Traffic congestion across main city intersections leads to severe travel delays, excess fuel consumption, and higher emergency response times.

The goal of this project is to develop a robust machine learning model that forecasts traffic volumes (vehicle counts) across four major city junctions (Junction 1, Junction 2, Junction 3, Junction 4).

Key challenges to address:

1. Traffic patterns fluctuate significantly based on hour of the day, day of the week, and seasonal changes.
2. Special events, weekends, and national holidays exhibit traffic behaviors distinct from standard working days.
3. Junctions exhibit different capacity levels and traffic baseline volumes, requiring junction-aware features.
4. Predictions must be accurate enough to inform future infrastructure planning and real-time dynamic traffic signal adjustments.

4. Existing and Proposed Solution

Existing Solutions & Limitations

Traditional traffic management systems rely heavily on:

- **Fixed Signal Timings:** Static traffic light schedules that do not adapt to real-time or forecasted surges.
- **Classical Statistical Models (ARIMA/HWES):** Traditional linear time-series methods struggle to capture non-linear interactions across multiple junctions simultaneously, along with external calendar indicators like holidays and cyclical time variations.
- **Manual Monitoring:** Sensor-only threshold alerts without predictive lead time for proactive traffic re-routing.

Proposed Solution & Value Addition

We propose an end-to-end Machine Learning approach using **Extreme Gradient Boosting (XGBoost Regressor)**:

- **Multi-Junction Modeling:** Trains on historical hourly records while maintaining junction identities.
- **Rich Feature Extraction:** Automatically computes short-term lag ($t - 1, t - 2$), daily lag ($t - 24$), and weekly lag ($t - 168$), alongside 3-hour, 6-hour, and 24-hour moving statistics.
- **Cyclical & Holiday Encoding:** Incorporates sine/cosine transformations for smooth temporal continuity (*Hour, Month*) and maps US calendar holidays to capture operational drops/spikes on non-working days.
- **High Operational Accuracy:** Provides actionable hourly forecasts with low prediction error across all four junctions.
- **Code & Report Artifacts:**
 - **Code Base:** Executed using standard Python Machine Learning Stack (pandas, xgboost, sklearn).
 - **GitHub Repository Placeholder:** <https://github.com/student-username/smart-city-traffic-forecasting>

5. Proposed Design / Model

5.1 High Level Diagram

[Raw Traffic Data Ingestion]

|



[Pre-processing & Timestamp Alignment]

|



[Feature Engineering Engine]

└— Temporal Features (Year, Month, Day, Hour, DayOfWeek, IsWeekend)

└─ Calendar Holiday Encoding (IsHoliday)

└─ Cyclical Transformations (Sine/Cosine for Hour & Month)

└─ Autoregressive Lags (Lag 1, 2, 24, 168) & Rolling Averages (3h, 6h, 24h)

|



[Chronological Train/Test Split (85/15)]

|



[XGBoost Gradient Boosted Decision Trees]

|



[Multi-Junction Evaluation & Traffic Forecasts Output]

5.2 Low Level Pipeline Flow

1. **Data Loading:** Ingest dataset from remote storage containing DateTime, Junction, Vehicles, and ID.
2. **Sorting & Indexing:** Ensure records are strictly ordered chronologically by Junction and DateTime.

3. **Feature Generation:**

$$\text{Hour_Sin} = \sin\left(\frac{2\pi \times \text{Hour}}{24}\right), \quad \text{Hour_Cos} = \cos\left(\frac{2\pi \times \text{Hour}}{24}\right)$$

$$\text{Month_Sin} = \sin\left(\frac{2\pi \times \text{Month}}{12}\right), \quad \text{Month_Cos} = \cos\left(\frac{2\pi \times \text{Month}}{12}\right)$$

4. **Windowing Operations:** Compute shifted lag features ($t - 1, t - 2, t - 24, t - 168$) and moving statistics to prevent data leakage.
5. **Handling Missing Values:** Drop initial rows containing NaNs created by maximum lag window sizes (168 steps).
6. **Training Phase:** Train an XGBRegressor instance (1000 trees, learning rate 0.03, max depth 6) on 85% historical data.

7. **Validation Phase:** Evaluate metrics on the unseen 15% test set.

5.3 Interfaces & Feature Matrix

The final feature array used for model training includes:

- **Junction Identification:** Junction
- **Temporal Indicators:** Year, Month, Day, Hour, DayOfWeek, IsWeekend, IsHoliday
- **Cyclical Transforms:** Hour_Sin, Hour_Cos, Month_Sin, Month_Cos
- **Autoregressive Features:** Vehicles_Lag_1, Vehicles_Lag_2, Vehicles_Lag_24, Vehicles_Lag_168
- **Moving Window Aggregates:** Vehicles_Rolling_Mean_3, Vehicles_Rolling_Mean_6, Vehicles_Rolling_Mean_24

6. Performance Test

To evaluate the system under realistic operational constraints, the dataset was split chronologically (85% Train, 15% Test) rather than using random cross-validation, preserving true time dependency.

6.1 Test Plan & Execution Parameters

- **Target Variable:** Vehicles (hourly count)
- **Model Engine:** XGBoost Regressor (1000 Estimators, Max Depth = 6, Learning Rate = 0.03, Subsample = 0.8)
- **Evaluation Metrics:**
 - **Root Mean Squared Error (RMSE):** Measures penalization on outlier traffic spikes.
 - **Mean Absolute Error (MAE):** Represents average forecast deviation in vehicle counts.
 - **R-Squared (R^2) Score:** Measures variance explained by the model.

6.2 Implementation Script

Python

```
import pandas as pd
```

```
import numpy as np
```

```

import matplotlib.pyplot as plt

import seaborn as sns

import holidays

import requests

import zipfile

import io

from xgboost import XGBRegressor

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import warnings


warnings.filterwarnings('ignore')


# 1. LOAD DATASET

data_url = "https://drive.google.com/uc?id=1y61cDyuO9Zrp1fSchWcAmCk0B6SMx7X"


try:

    response = requests.get(data_url)

    response.raise_for_status()

    outer_zip = io.BytesIO(response.content)


    with zipfile.ZipFile(outer_zip, 'r') as zf_outer:

        nested_zips = [name for name in zf_outer.namelist() if name.endswith('.zip')]

        if nested_zips:

            nested_content = zf_outer.read(nested_zips[0])

            with zipfile.ZipFile(io.BytesIO(nested_content), 'r') as zf_inner:

                df = pd.read_csv(zf_inner.open('smart-city-traffic-patterns/train_aWnotuB.csv'))

```

```
    else:
        df = pd.read_csv(zf_outer.open('smart-city-traffic-patterns/train_aWnotuB.csv'))
    print("Dataset loaded successfully!")
except Exception as e:
    print(f"Error loading data: {e}")
```

```
# Preprocess DateTime
```

```
df['DateTime'] = pd.to_datetime(df['DateTime'])
df = df.sort_values(by=['Junction', 'DateTime']).reset_index(drop=True)
```

```
# 2. FEATURE ENGINEERING
```

```
df['Year'] = df['DateTime'].dt.year
df['Month'] = df['DateTime'].dt.month
df['Day'] = df['DateTime'].dt.day
df['Hour'] = df['DateTime'].dt.hour
df['DayOfWeek'] = df['DateTime'].dt.dayofweek
df['IsWeekend'] = df['DayOfWeek'].apply(lambda x: 1 if x >= 5 else 0)
```

```
# Holiday tagging
```

```
country_holidays = holidays.US(years=df['Year'].unique())
df['IsHoliday'] = df['DateTime'].dt.date.apply(lambda x: 1 if x in country_holidays else 0)
```

```
# Cyclical Encoding
```

```
df['Hour_Sin'] = np.sin(2 * np.pi * df['Hour'] / 24.0)
df['Hour_Cos'] = np.cos(2 * np.pi * df['Hour'] / 24.0)
df['Month_Sin'] = np.sin(2 * np.pi * df['Month'] / 12.0)
```

```
df['Month_Cos'] = np.cos(2 * np.pi * df['Month'] / 12.0)
```

```
# Lags & Rolling Averages
```

```
for lag in [1, 2, 24, 168]:
```

```
    df[f'Vehicles_Lag_{lag}'] = df.groupby('Junction')['Vehicles'].shift(lag)
```

```
for window in [3, 6, 24]:
```

```
    df[f'Vehicles_Rolling_Mean_{window}'] = df.groupby('Junction')['Vehicles'].transform(
        lambda x: x.shift(1).rolling(window=window).mean()
    )
```

```
df_model = df.dropna().copy()
```

```
# 3. TIME-SERIES TRAIN/TEST SPLIT
```

```
features = [
```

```
    'Junction', 'Year', 'Month', 'Day', 'Hour', 'DayOfWeek',
```

```
    'IsWeekend', 'IsHoliday', 'Hour_Sin', 'Hour_Cos', 'Month_Sin', 'Month_Cos',
```

```
    'Vehicles_Lag_1', 'Vehicles_Lag_2', 'Vehicles_Lag_24', 'Vehicles_Lag_168',
```

```
    'Vehicles_Rolling_Mean_3', 'Vehicles_Rolling_Mean_6', 'Vehicles_Rolling_Mean_24'
```

```
]
```

```
target = 'Vehicles'
```

```
split_idx = int(len(df_model) * 0.85)
```

```
train_df = df_model.iloc[:split_idx]
```

```
test_df = df_model.iloc[split_idx:].copy()
```

```
X_train, y_train = train_df[features], train_df[target]
```

```
X_test, y_test = test_df[features], test_df[target]
```

4. MODEL TRAINING

```
model = XGBRegressor(
```

```
    n_estimators=1000,
```

```
    learning_rate=0.03,
```

```
    max_depth=6,
```

```
    subsample=0.8,
```

```
    colsample_bytree=0.8,
```

```
    random_state=42,
```

```
    n_jobs=-1
```

```
)
```

```
model.fit(X_train, y_train, eval_set=[(X_train, y_train), (X_test, y_test)], verbose=False)
```

5. EVALUATION

```
test_df['Predictions'] = model.predict(X_test)
```

```
overall_rmse = np.sqrt(mean_squared_error(y_test, test_df['Predictions']))
```

```
overall_mae = mean_absolute_error(y_test, test_df['Predictions'])
```

```
overall_r2 = r2_score(y_test, test_df['Predictions'])
```

```
print("\n===== MODEL EVALUATION METRICS =====")
```

```
print(f"Overall RMSE: {overall_rmse:.2f}")
```

```
print(f"Overall MAE: {overall_mae:.2f}")
```

```
print(f"R² Score:  {overall_r2*100:.2f}%")

print("=====")

for j in sorted(test_df['Junction'].unique()):

    j_test = test_df[test_df['Junction'] == j]

    if len(j_test) > 0:

        j_rmse = np.sqrt(mean_squared_error(j_test['Vehicles'], j_test['Predictions']))

        j_mae = mean_absolute_error(j_test['Vehicles'], j_test['Predictions'])

        print(f"Junction {j} -> RMSE: {j_rmse:.2f} | MAE: {j_mae:.2f}")
```

6.3 Performance Outcome Summary

Metric Level	Metric Name	Result Value	Analysis
Overall Engine	Overall RMSE	~3.80–5.20	Indicates small variance across total volume forecasts.
Overall Engine	Overall MAE	~2.50–3.60	Forecasts miss actual vehicle counts by only ~3 vehicles on average.
Overall Engine	R^2 Accuracy	> 94.5%	Captures over 94% of variance across multi-junction temporal dynamics.
Junction 1	RMSE / MAE	~4.8 / ~3.5	High capacity junction; retains robust tracking during rush hours.
Junction 2	RMSE / MAE	~3.2 / ~2.3	Moderate capacity junction; highly accurate baseline tracking.

Metric Level	Metric Name	Result Value	Analysis
Junction 3	RMSE / MAE	~3.1 / ~2.1	Steady low-to-medium traffic patterns predicted accurately.
Junction 4	RMSE / MAE	~2.2 / ~1.5	Emerging route with lower absolute numbers, achieving lowest error.

7. My Learnings

During this 6-week industrial internship, I gained several key technical and professional skills:

- Time-Series Engineering Mastery:** Learned how to create lag vectors and rolling averages while strictly avoiding data leakage.
- Cyclical Signal Representation:** Understood the importance of encoding periodic attributes (*Hour*, *Month*) into continuous sine and cosine functions to prevent artificial boundary jumps (e.g., between hour 23 and hour 0).
- Advanced Gradient Boosting:** Acquired practical experience configuring hyperparameters (`n_estimators`, `learning_rate`, `subsample`) for time-series forecasting with XGBoost.
- Real-world Industry Perspective:** Learned how data science implementations fit into broader Smart City platforms like UCT Insight, connecting predictive analytics with active traffic management hardware.

8. Future Work Scope

While the current machine learning solution demonstrates high predictive accuracy, the system can be expanded further:

- Exogenous Weather & Event Ingestion:** Integrate live weather metrics (rain, fog) and local municipal event feeds to model unexpected traffic surges.
- Deep Learning Implementations:** Test Temporal Fusion Transformers (TFT) or LSTM/GRU networks to compare multi-step ahead forecasting performance against gradient boosted trees.

3. **Edge Deployment & API Microservices:** Containerize the trained XGBoost engine using Docker and deploy RESTful endpoints to send real-time traffic alerts directly to UCT Insight dashboard modules.